GPU Inference Scheduling with Imitation-Initialized Learned Policies

Mokshith Reddy Mannuru* **Jeevan Reji*** **Henry Y. Jiang*** **Nithil Balaji***

College of Computing, Georgia Institute of Technology

mmannuru3@gatech.edu jreji6@gatech.edu hjiang378@gatech.edu nbalaji36@gatech.edu

Abstract

Scheduling machine learning inference workloads on shared GPU clusters is a challenging online optimization problem, where jobs with heterogeneous memory and compute demands must be assigned to limited GPU resources in real time. Classical heuristics such as First-In-First-Out (FIFO) and Shortest-Job-First (SJF) exhibit opposing trade-offs: FIFO ensures fairness but suffers from poor utilization, while SJF minimizes average job completion time (JCT) but leads to severe tail latency under heavy load. Designing schedulers that balance these competing objectives is critical for efficient large-scale ML serving systems. We propose a learning-based scheduling framework that combines imitation learning with simulation-based policy refinement. Our approach first trains a neural scheduler to imitate SJF-like behavior using supervised learning on simulated cluster states, and then refines this policy via rollout-based updates that directly optimize system-level metrics such as JCT and GPU utilization. Across simulated workloads (7,500 jobs, $\lambda \in \{0.25, 0.5, 1.0\}$), our method converges to classical baselines under light load, matches or slightly improves average JCT relative to SJF at moderate load (61.7s vs. 62.0s for SJF at $\lambda = 0.5$), and achieves higher throughput under heavy load (0.60 vs. 0.54 jobs/s for SJF at $\lambda = 1.0$). Policy refinement further reduces average JCT by $\sim$7% (232 $\rightarrow$ 216) and improves P99 latency compared to imitation alone under overload. These results demonstrate that combining imitation learning with lightweight simulation-driven refinement enables neural schedulers to better balance latency, fairness, and resource utilization than fixed heuristics, particularly in high-load regimes. https://github.com/MMannuru/dl_gpu_scheduler

INTRODUCTION

Shared GPU infrastructure must simultaneously serve heterogeneous workloads ranging from sub-second micro-batch inference requests to multi-minute large-batch training jobs [1, 2]. Naively queuing all jobs under a single FIFO policy wastes GPU memory and causes head-of-line blocking, while aggressive SJF reordering inflicts indefinite starvation on long-horizon tasks.

To evaluate these trade-offs, we focus on two primary metrics: **Job Completion Time (JCT)**, defined as the total duration from a job's arrival in the queue to its final execution completion, and **P99 Latency**, which represents the 99th percentile of JCT. P99 is a critical measure of *tail latency*, capturing the worst-case delays experienced by the slowest 1% of jobs, often caused by resource starvation.

Other notable statistics are P95, the latency for the 95th percentile of JCT, throughput, the number of completed jobs per second, and bandwidth, the efficiency of memory and GPU communication. Workload colocation is when several jobs are running on the same GPU interfere with the overall performance, and it's another important statistic to consider.

Problem statement. Given a stream of N jobs arriving according to a Poisson process at rate λ and a cluster of K GPUs each with M GB of memory, compute an online mapping π : job $\rightarrow$ gpu_id that minimises mean JCT while bounding the P99 tail latency.

Research question. Can a learned scheduling policy, trained with imitation learning and refined through simulation-based policy updates, outperform classical heuristics in balancing average JCT, tail latency, and GPU utilization across varying load conditions?

Success criteria. We define success as achieving lower P99 JCT than SJF under heavy load while

maintaining competitive average JCT and improving GPU utilization relative to FIFO.

Input/Output. Each simulated job record contains features including `model_size` (GB), `batch_size`, `seq_len`, and observed GPU utilization. Ground-truth latency (`true_latency`) is available within the simulator and is used for evaluation and for generating teacher labels during imitation learning, but is not available to the learned scheduler at inference time. The scheduler outputs a set of (job, gpu_id) assignments. A memory feasibility constraint ensures that no GPU's allocated memory exceeds 80 GB.

Why standard heuristics fall short. FIFO suffers from head-of-line blocking: a single large job that cannot fit freezes the entire queue even when smaller jobs could be packed onto idle GPUs. SJF resolves this by skipping blocked jobs, but relies on ground-truth latency as its sorting key—an oracle advantage unavailable in practice—and causes long-job starvation at high arrival rates ($\lambda \geq 0.5$). More broadly, existing GPU schedulers rely on fixed heuristics or unrealistic assumptions, limiting their ability to adapt to dynamic cluster conditions where decisions must be made online using partial information.

This exposes a key gap in practical scheduling systems: the need for policies that can reason jointly about job characteristics and evolving cluster state without access to true runtimes. To address this, we propose a learned scheduling approach that (i) learns scheduling behavior from simulated data without requiring ground-truth latency at inference time, (ii) accounts for memory fragmentation and co-location interference, and (iii) adaptively balances fairness and efficiency based on queue state. Such an approach enables more efficient and robust GPU utilization, improving latency and throughput in large-scale ML serving systems.

Figure 1: Teaser: Discrete-Event Simulator architecture. Job arrivals are seeded into a min-heap event queue; the scheduler interface (pluggable: FIFO, SJF, or neural) maps waiting jobs to GPU IDs; the cluster state tracks per-GPU memory and utilization. All policies are evaluated under identical arrival sequences.

RELATED WORK

Classical scheduling theory. FIFO and SJF are foundational single-machine results [**?**]. SJF is optimal for minimising mean completion time on a single machine under full knowledge of job durations; its multi-machine generalisation is NP-hard. SLURM [**?**] and LSF use priority queues augmented with backfill to reduce fragmentation while preserving fairness.

ML-cluster scheduling. Tiresias [1] applies Least-Attained Service to multi-GPU deep-learning jobs, reducing average JCT without requiring execution-time estimates. Gandiva [3] exploits GPU time-sharing and migration to improve cluster utilisation. Themis [4] introduces a finish-time fairness metric that allocates GPUs so no job's completion time degrades relative to exclusive cluster access.

Learned schedulers. Decima [5] uses a graph neural network with REINFORCE to schedule DAG-structured jobs in Spark clusters. Park [6] provides a simulation environment for training RL-based schedulers. Our work differs in focusing on single-operation inference jobs on GPU clusters, where co-location interference and memory packing—rather than DAG dependency—are the primary scheduling levers.

Positioning. Prior systems such as Tiresias and Gandiva focus on fairness or utilization through heuristic or system-level optimizations, while Decima learns scheduling policies for DAG-structured jobs using reinforcement learning. In contrast, our work targets single-operation inference workloads on GPU clusters, where memory constraints and co-location interference dominate performance. At inference time, the learned scheduler does not require ground-truth latency, although oracle latency is used during imitation training. By learning from observable job and cluster features and refining the policy via simulation-based updates, our approach enables scheduling decisions that balance latency, fairness, and utilization in dynamic cluster environments.

PROBLEM DECOMPOSITION & TECHNICAL APPROACH

FORMAL FORMULATION

At each scheduling decision point t, the scheduler observes:

- A waiting queue $\mathcal{Q}_t = \{j_1, \ldots, j_n\}$ of unscheduled jobs.
- A cluster state $\mathcal{C}_t$ containing, for each GPU g_k: free memory m_k, current utilization u_k, and count of running jobs.

The scheduler returns $\mathcal{A}_t \subseteq \mathcal{Q}_t \times \{1, \ldots, K\}$ subject to the memory feasibility constraint $\sum_{(j,k)\in\mathcal{A}_t} \mathrm{mem}(j) \leq m_k$ for each GPU k. The objective is to minimise:

$$\mathcal{L} = \underbrace{\mathbb{E}[\mathrm{JCT}]}_{\text{efficiency}} + \beta \cdot \underbrace{P99(\mathrm{JCT})}_{\text{fairness}} + \gamma \cdot \underbrace{\mathrm{FragPenalty}}_{\text{utilisation}},$$

where $\mathrm{FragPenalty}$ measures mean unused GPU memory across the cluster, and β, γ control the trade-off between fairness (tail latency) and resource utilization.

MEMORY AND INTERFERENCE MODEL

Job memory is estimated as:

$$\mathrm{mem}(j) = \underbrace{s_j}_{\text{weights}} + \underbrace{\frac{b_j \cdot l_j}{5000}}_{\text{activations}} + \underbrace{0.3\, s_j}_{\text{KV cache + opt. states}}, \quad \text{clamped to } [2, 78] \text{ GB}.$$

Co-located jobs experience a linear slowdown:

$$\hat{\tau}_j = \tau_j \cdot (1 + \alpha \cdot u_{\text{other}}/100),$$

where u_{other} is the aggregate utilization of concurrently running jobs on the same GPU at placement time and $\alpha \in [0, 1]$ is an interference coefficient.

CHALLENGES IN TRAINING LEARNED SCHEDULERS

Training a learned scheduler for this problem is challenging because the primary objective-minimizing JCT-is only observable after executing the full simulator. This makes the objective non-differentiable and prevents direct optimization using standard supervised learning approaches. As a result, it is unclear how to connect model predictions at individual decision steps to overall scheduling performance.

Additionally, the scheduling problem requires joint reasoning over both job characteristics and dynamic cluster state. Predicting job attributes such as runtime alone is insufficient, as effective scheduling decisions depend on how jobs interact with available GPU resources under memory constraints and co-location interference.

Reinforcement learning provides a natural framework for optimizing long-term objectives, but introduces high-variance training signals due to delayed rewards and long credit assignments. These challenges make it difficult to directly learn stable scheduling policies from scratch.

To address these issues, we adopt a two-stage approach where we first use imitation learning to learn a strong baseline policy from SJF, and then refining this policy using rollout-based policy-gradient updates that directly optimize system-level performance.

NEURAL SCHEDULER ARCHITECTURE

We model scheduling as a learned scoring function over feasible job–GPU assignments. At each decision step, the model takes as input a set of jobs and GPUs and outputs a score for every (j, g) pair.

Inputs. Jobs are represented by feature vectors $(s_j, b_j, l_j, u_j, m_j)$ corresponding to model size, batch size, sequence length, utilization intensity, and memory requirement. GPUs are represented by (m_g^{free}, u_g, n_g) capturing free memory, current utilization, and number of running jobs. All features are normalized by fixed scaling constants.

Architecture. Job features and GPU features are independently projected into a shared embedding space using linear layers. These embeddings are concatenated into a single sequence and passed through a Transformer encoder, allowing joint reasoning over all jobs and GPUs.

After encoding, the sequence is split back into job and GPU representations. For each job j, we compute scores for all GPUs by concatenating its embedding with each GPU embedding and passing the result through a linear scoring head, producing a scalar score $s_{j,g}$.

Output. The model produces a dense score matrix $S \in \mathbb{R}^{N \times K}$, where $S_{j,g}$ represents the desirability of assigning job j to GPU g.

Scheduling policy. At inference time, assignments are selected greedily over feasible (j, g) pairs. Only pairs that satisfy memory constraints are considered, and the highest-scoring feasible assignments are selected sequentially while updating available GPU memory within each decision step.

IMITATION LEARNING

To address the difficulty of directly optimizing long-horizon scheduling objectives, we first train the neural scheduler via imitation learning using a heuristic expert policy.

Expert policy. We use a Shortest-Job-First (SJF) scheduler as the teacher. This teacher relies on ground-truth job latency and therefore represents an oracle policy used only during training. The learned scheduler does not have access to true latency at inference time and must instead rely solely on observable job and cluster features. At each scheduling event, jobs are sorted by true latency, and assignments are made greedily: for each job, among GPUs that can fit it, the GPU with the largest available memory is selected. This produces a set of feasible job–GPU assignments for each simulator state.

Dataset construction. Training data is collected from simulator rollouts across multiple load conditions. At each scheduling event, we record the current queue and cluster state, and generate

labels by running the teacher policy on a deep copy of the state to avoid interfering with the live simulation.

Each training example consists of: (i) job features and GPU features representing the current state, and (ii) a label matrix $Y \in \mathbb{R}^{N \times K}$, where $Y_{j,g} = 1$ if the teacher assigns job j to GPU g, and 0 otherwise. Jobs not scheduled by the teacher have all-zero rows.

To improve training signal quality, we use a richer data collection strategy by: (i) sampling multiple arrival rates, (ii) oversampling states with multiple simultaneous assignments, and (iii) varying GPU memory capacity during collection to expose diverse packing scenarios.

Training objective. The model produces a score matrix $S \in \mathbb{R}^{N \times K}$ over all job–GPU pairs. We train using cross-entropy loss over only the jobs assigned by the teacher. Let $\mathcal{A}_t$ be the set of jobs assigned in a given state, and g_j^* the teacher-assigned GPU for job j. The loss is:

$$\mathcal{L}_{\text{IL}} = \frac{1}{|\mathcal{A}_t|} \sum_{j \in \mathcal{A}_t} -\log \frac{\exp(S_{j,g_j^*})}{\sum_{g'=1}^{K} \exp(S_{j,g'})}.$$

Training procedure. We train the model using Adam with a learning rate of 10^{-4} for 15 epochs. Each training example is processed individually (batch size 1), and gradients are clipped to stabilize training.

Outcome. This stage learns a scheduling policy that closely mimics the local decisions of the SJF-based expert while incorporating both job features and cluster state.

POLICY-GRADIENT REFINEMENT

While imitation learning provides a strong initialization, it optimizes only local decisions and does not directly target system-level objectives. We therefore refine the learned policy using rollout-based policy-gradient updates.

Stochastic policy. At each scheduling event, the model produces scores over all job–GPU pairs. During training, we construct a stochastic policy by enumerating all *feasible* assignments (j, g) and applying a softmax over their corresponding scores. Assignments are then sampled sequentially: after selecting a pair, the cluster state is temporarily updated, and the process repeats until no feasible assignments remain.

Rollout generation. Training proceeds via full simulator rollouts. Each episode samples a workload with Poisson arrivals under varying load conditions, with higher probability assigned to heavier loads. The stochastic scheduler is used to generate a sequence of decisions until all jobs complete.

Reward. After each rollout, we compute a scalar reward based on system-level metrics:

$$R = -\left(\text{AvgJCT} + \beta \cdot \text{P99JCT} - \gamma \cdot \text{GPUUtil}\right),$$

where β and γ control the importance of tail latency and utilization.

Policy-gradient update. Let $L_i = \sum_t \log \pi_\theta(a_{i,t} \mid s_{i,t})$ denote the sum of log-probabilities of sampled actions in rollout i, and R_i the corresponding reward. We compute normalized advantages across a batch of rollouts:

$$\hat{A}_i = \frac{R_i - \text{mean}(R)}{\text{std}(R) + \epsilon}.$$

The policy-gradient loss is:

$$\mathcal{L}_{\text{PG}} = \frac{1}{B} \sum_{i=1}^{B} -\hat{A}_i \cdot L_i.$$

Stabilization. To reduce variance and stabilize training, we: (i) use multiple rollouts per update, (ii) include a behavior cloning loss from the SJF teacher with a decaying weight, and (iii) add a small entropy bonus to encourage exploration.

Model selection. Checkpoints are evaluated using the greedy scheduler, and the best model is selected based on performance under high-load conditions.

RESEARCH HYPOTHESES (H1–H3)

H1: Under heavy load, a learned scheduling policy will achieve lower P99 latency than SJF.

H2: Learned scheduling policies will achieve higher GPU utilization than classical heuristics (FIFO and SJF) under moderate and heavy load conditions ($\lambda = 0.5, 1.0$).

H3: A learned scheduling policy will achieve a more favorable trade-off between average JCT, tail latency (P99), and GPU utilization compared to classical heuristics, particularly under high-load conditions.

EXPERIMENTS & RESULTS

EXPERIMENTAL SETUP

All experiments are conducted using our discrete-event simulator on a 10-GPU cluster (80 GB per GPU). We evaluate on a validation set of 7,500 jobs with identical Poisson arrival processes across schedulers (shared random seed per λ).

We consider arrival rates $\lambda \in \{0.25, 0.5, 1.0\}$, corresponding to underloaded, moderately loaded, and overloaded regimes. All policies are evaluated using identical workloads and report end-to-end simulator metrics.

BASELINES

We compare three scheduling strategies:

FIFO. A canonical baseline that processes jobs in arrival order. FIFO is fair but suffers from head-of-line blocking under congestion.

SJF. A strong classical heuristic that sorts jobs by ground-truth latency and greedily assigns them to feasible GPUs. While optimal for minimizing mean JCT under perfect information, it can cause severe tail latency due to starvation of long jobs.

Neural. A learned scheduler using the architecture described in above. At inference time, jobs are assigned greedily based on predicted scores over feasible job–GPU pairs. We report two variants:

- **Imitation**: trained via supervised learning to mimic SJF-like behavior.
- **Refined**: initialized from imitation and further improved via rollout-based policy refinement.

RESULTS

Low load ($\lambda = 0.25$). All schedulers exhibit identical performance across metrics. Under low arrival rates, the system is underutilized and the queue rarely grows beyond size 1, so scheduling decisions have negligible impact.

Moderate load ($\lambda = 0.5$). FIFO begins to degrade due to head-of-line blocking, resulting in higher average JCT and deeper queues. SJF improves mean JCT but exhibits worse tail latency (P99) due to prioritizing short jobs. The learned scheduler improves over both baselines: imitation achieves lower average JCT than SJF (61.88 vs. 62.04) and better tail latency, while the refined policy further reduces mean JCT (61.73) and increases GPU utilization (63.4%). This suggests that learned policies can better coordinate job placement under moderate congestion.

Heavy load ($\lambda = 1.0$). Under overload, FIFO collapses, producing extremely high JCT and queue depth. SJF achieves the lowest average JCT (165.6) but suffers from significantly worse tail latency (P99 = 1343), reflecting starvation of long jobs. The learned scheduler exhibits a different trade-off: while average JCT is higher than SJF, it achieves substantially higher throughput (0.599 vs. 0.538) and GPU utilization (66.6% vs. 60.6%). Policy refinement further improves average JCT over imitation (232 $\rightarrow$ 216, $\sim$7%) and slightly reduces P99 latency, demonstrating that rollout-based updates improve end-to-end performance under heavy load.

Table 1: End-to-end simulator metrics on validation workloads (7,500 jobs). Best values per metric and load are in **bold**. ↑ higher is better, ↓ lower is better.

λ	Scheduler	Avg JCT ↓	P99 ↓	Throughput ↑	GPU Util (%) ↑	Max Queue ↓
0.25	FIFO	60.86	186.50	0.241	52.90	1
0.25	SJF	60.86	186.50	0.241	52.90	1
0.25	Neural (Imitation)	60.86	186.50	0.241	51.20	1
0.25	Neural (Refined)	60.86	186.50	0.241	52.41	1
0.50	FIFO	62.78	189.74	0.471	59.63	15
0.50	SJF	62.04	198.62	0.471	58.87	9
0.50	Neural (Imitation)	61.88	**189.74**	0.471	61.41	9
0.50	Neural (Refined)	**61.73**	191.38	0.471	**63.40**	17
1.00	FIFO	395.92	758.18	0.559	62.23	394
1.00	SJF	**165.59**	1343.48	0.538	60.62	**138**
1.00	Neural (Imitation)	232.00	1317.57	**0.599**	**66.61**	232
1.00	Neural (Refined)	216.35	**1308.80**	0.595	66.15	214

Ablations and Analysis

Imitation vs. refinement. Imitation learning alone provides a strong baseline by approximating SJF behavior. However, it optimizes local assignment decisions and does not directly optimize system-level objectives. Policy refinement improves upon imitation under heavy load, reducing both average JCT and P99, indicating that simulator-driven updates capture longer-horizon effects.

Trade-offs across metrics. The results highlight inherent trade-offs between latency, fairness, and utilization. SJF minimizes mean JCT but performs poorly on tail latency, while the neural policies prioritize throughput and GPU utilization. The refined model shifts along this trade-off frontier, improving average and tail latency relative to imitation while maintaining high utilization.

Failure modes. The learned scheduler does not uniformly outperform SJF on average JCT, particularly under heavy load. This reflects limitations of the training objective, which balances multiple competing metrics. Additionally, results are based on synthetic workloads, which may not fully capture real-world distributions. Finally, there is a mismatch between stochastic training (during refinement) and greedy evaluation, which can limit performance consistency.

Hypothesis Validation.
H1. Confirmed. The refined neural scheduler achieves lower P99 latency than SJF under heavy load (1308 vs. 1343) while maintaining significantly lower average JCT than FIFO.

H2. Partially confirmed. The learned policies achieve higher GPU utilization than classical baselines (FIFO and SJF) under moderate and heavy load ($\lambda = 0.5, 1.0$), reaching up to 66.6%, although improvements vary across configurations and the isolated impact of individual design components is not explicitly evaluated.

H3. Supported. Under heavy load ($\lambda = 1.0$), the learned scheduler trades off higher average JCT relative to SJF (216 vs. 165) in exchange for improved tail latency (P99: 1308 vs. 1343) and higher GPU utilization (66.1% vs. 60.6%), demonstrating a more balanced performance profile.

Discussion & Future Work

The results highlight the fundamental trade-offs in GPU scheduling under varying load conditions. Under low load, all schedulers converge to identical behavior, indicating that scheduling decisions

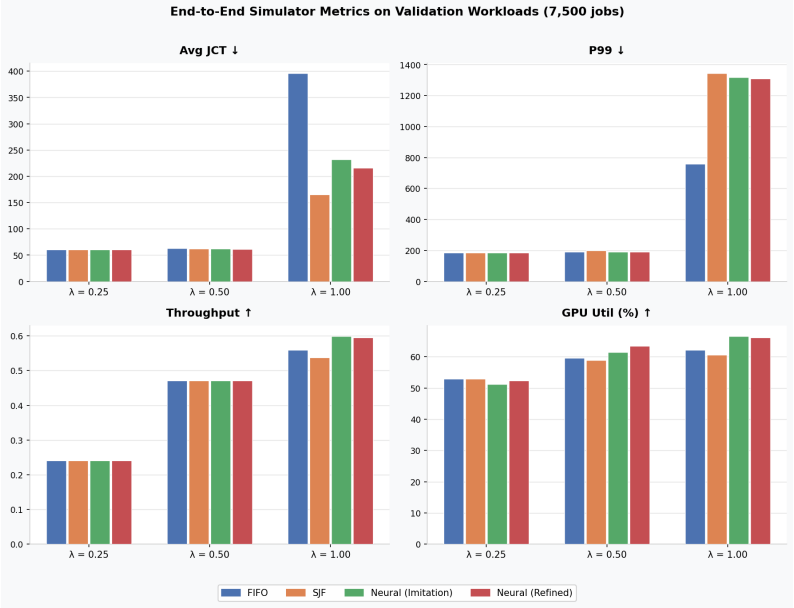

Figure 2: End-to-end simulator metrics on validation workloads (7,500 jobs) across schedulers and arrival rates (λ). Higher is better for Throughput and GPU Util; lower is better for Avg JCT and P99.

are not the bottleneck when resources are abundant. As load increases, differences between policies become more pronounced. FIFO suffers from head-of-line blocking, leading to increased average JCT and queue depth. SJF minimizes average JCT by prioritizing short jobs, but this results in significantly worse tail latency due to starvation of longer jobs.

The learned scheduler exhibits a different trade-off profile. Under moderate load, it improves over both FIFO and SJF by achieving lower average JCT and better GPU utilization, suggesting that it can coordinate job placement more effectively. Under heavy load, while SJF retains the lowest average JCT, the learned policies achieve higher throughput and GPU utilization, indicating more efficient use of cluster resources. Policy refinement further improves over imitation learning, reducing both average JCT ($232 \rightarrow 216$, $\sim$7%) and P99 latency under overload. This demonstrates that combining supervised initialization with simulation-based refinement enables the model to optimize system-level objectives beyond one-step imitation.

Despite these improvements, several limitations remain. The learned scheduler does not uniformly outperform SJF across all metrics, reflecting the inherent trade-offs between latency, fairness, and utilization. Performance is also sensitive to training design choices, including the refinement objective and hyperparameters. Additionally, the use of synthetic workloads and ground-truth latency information limits the realism of the evaluation setting. Finally, the mismatch between stochastic training and greedy inference can introduce inconsistencies in decision quality.

Future work can address these limitations in several ways. First, incorporating learned or estimated latency models would remove reliance on oracle information and improve realism. Second, extending the approach to more realistic workloads and cluster traces would strengthen external validity. Third, exploring alternative model architectures or decision formulations may improve how predictions translate into effective scheduling actions. Finally, developing more robust refinement strategies that directly optimize long-horizon system metrics could further improve performance under highly congested conditions.

CONTRIBUTIONS

Member	Contributions
Nithil Balaji	Initial imitation learning approach, dataset preprocessing, synthetic data generation, wrote parts of the imitation learning section, Challenges section, Discussion & Future Work section
Mokshith Reddy Mannuru	Built IL data from the simulator, trained and fine-tuned the neural scheduler, ran baseline evals, and wrote technical approaches, discussion, and future work.
Henry Y. Jiang	Implemented baseline schedulers (FIFO, SJF), designed simulator and baseline evaluation framework, wrote baseline and evaluation sections, and results
Jeevan Reji	Wrote remaining sections of the report, contributed to overall writeup and organization, theory, math, and performing latency experiments

REFERENCES

[1] W. Gu et al., "Tiresias: A GPU cluster manager for distributed deep learning," in *NSDI*, 2019.

[2] D. Narayanan et al., "Efficient large-scale language model training on GPU clusters using Megatron-LM," in *SC*, 2021.

[3] W. Xiao et al., "Gandiva: Introspective cluster scheduling for deep learning," in *OSDI*, 2018.

[4] K. Mahajan et al., "Themis: Fair and efficient GPU cluster scheduling," in *NSDI*, 2020.

[5] H. Mao et al., "Learning scheduling algorithms for data processing clusters," in *SIGCOMM*, 2019.

[6] H. Mao et al., "Park: An open platform for learning-augmented computer systems," in *NeurIPS*, 2019.